

Universidade Federal do Paraná

Departamento de Informática

CI1002 Programação 2

Conversão de Tipos e Uniões

Diego Addan

UFPR - 2026

Conversão de Tipos

Muitas vezes, expressões lógicas/aritméticas envolvem **valores de tipos distintos**, como char, int, float e double, que devem ser **convertidos** durante o cálculo.

Algumas conversões são efetuadas implicitamente pelo compilador, sem riscos para a integridade dos dados manipulados. **Todavia, certas conversões podem levar a resultados errados ou à perda de informação e precisam ser “forçadas” pelo programador, através de conversões explícitas.**

Conversão de Tipos

Conversão implícita

Quando uma operação lógica/aritmética envolve dois operandos de tipos distintos, o compilador primeiro os converte para um único tipo, antes de avaliar a operação. Nos caso de tipos numéricos, essa conversão é denominada **promoção automática**

Na promoção automática, o tipo de menor tamanho (com menos bytes) é automaticamente convertido (“promovido”) para o tipo de maior tamanho.

char < short < int < long < long long < float < double < long double

Conversão de Tipos

Conversão implícita

```
char  c = 'A' ;
int   i = 3 ;
short s = 31 ;
float f = 31.8 ;
double d ;

i = c + 30 ;    // o valor de c (65) é convertido de char para int

d = i * f + s ; // ao avaliar i*f,      i é promovido de int a float
                // ao avaliar float + s, s é promovido de short a float
```

Conversão de Tipos

Conversão implícita

A conversão implícita (ou automática) é realizada para as operações aritméticas (+ - * / %), relacionais (< <= > >= == !=) e com bits (& | ^).

Os operadores de atribuição (=, +=, ...) também recebem uma conversão implícita de tipo, para compatibilizar o valor calculado (lado direito da atribuição) com a variável que irá recebê-lo (lado esquerdo).

Erros de conversão

A regra de promoção automática é aplicada a cada operação da expressão lógica/aritmética. Por isso, deve ser tomado cuidado em expressões mais complexas:

```
#include <stdio.h>

int main ()
{
    int i    = 100 ;
    int j    = 6   ;
    float x  = 6   ;
    float y  ;

    y = i / j * x ;           // (int / int) * float -> int * float
    printf ("y vale %f\n", y) ; // ^^^^^^^^^^^ divisão inteira!

    y = x * i / j ;           // (float * int) / int
    printf ("y vale %f\n", y) ; // resultado correto (vide abaixo)
}
```

Precedência de operadores.

Podemos garantir o resultado correto!

```
y = (x * i) / j ;
printf ("y vale %f\n", y) ;
```

Erros de conversão

A conversão implícita na atribuição também pode provocar perda de informação. O exemplo abaixo mostra algumas dessas situações:

```
int main ()
{
    char  c ;
    int   i ;
    long  l ;
    float f ;
    double d ;

    c = 'A' ;
    i = c ;           // ok, pois int > char
    printf ("c = %d, i = %d\n", c, i) ;

    i = 34 ;
    f = i ;           // ok, pois float > int
    printf ("i = %d, f = %f\n", i, f) ;

    l = 214748364347243 ;
    f = l ;           // perda de precisão
    printf ("l = %ld, f = %f\n", l, f) ;
}
```

```
l = 214748364347243 ;
d = l ;               // ok, precisão suficiente
printf ("l = %ld, d = %lf\n", l, d) ;

f = 451.28 ;
i = f ;               // parte fracionária é truncada
printf ("f = %f, i = %d\n", f, i) ;

d = 3.141592653589793264 ;
f = d ;               // perda de precisão
printf ("d = %.15f, f = %.15f\n", d, f) ;

l = 12345677890 ;
i = l ;               // perda dos bits mais significativos
printf ("l = %ld, i = %d\n", l, i) ;
}
```

Conversão explícita

Em alguns casos, é necessário forçar a conversão de tipos de dados, para que uma expressão seja avaliada da forma correta.

Podemos adicionar um elemento neutro de tipo float à expressão;

Ou
forçar a avaliação de **soma** ou **num** como **float** (type casting).

```
#include <stdio.h>

int main ()
{
    int soma, num ;
    float media ;

    soma = 10 ;
    num  = 4 ;

    media = soma / num ;

    printf ("media = %f\n", media) ;
}
```


Conversão explícita

```
#include <stdio.h>

int main ()
{
    int soma, num ;
    float media ;

    soma = 10 ;
    num  = 4 ;

    media = soma / num ;           // errado, perda de precisão
    printf ("media = %f\n", media) ;

    media = 1.0 * soma / num ;     // soma é "promovida" a float
    printf ("media = %f\n", media) ;

    media = (float) soma / num ;   // soma é avaliada como float (casting)
    printf ("media = %f\n", media) ;

    media = soma / (float) num ;   // num é avaliado como float (casting)
    printf ("media = %f\n", media) ;
}
```

Conversão de ponteiros

Uma operação frequente em C é a conversão de **tipos de ponteiros**. Muitas funções importantes, como **qsort** e **bsearch**, usam ponteiros genéricos **void*** como parâmetros de entrada, para poder receber dados de diversos tipos.

Como ponteiros para void não apontam para nenhum tipo válido de dado, eles não podem ser desreferenciados (ou seja, não é possível acessar diretamente os dados que eles apontam).

Por isso, **ponteiros para void precisam ser convertidos em ponteiros para algum tipo válido** antes de serem desreferenciados.

Conversão de ponteiros

```
#include <stdio.h>
#include <stdlib.h>

#define VETSIZE 10

int vetor[VETSIZE] ;

// compara dois inteiros apontados por "a" e "b"
int compara_int (const void* a, const void* b)
{
    int *pa, *pb ;

    pa = (int*) a ; // "vê" a como int*
    pb = (int*) b ; // idem, b

    if (*pa > *pb) return 1 ;
    if (*pa < *pb) return -1 ;
    return 0 ;
}
```

```
int main ()
{
    int i ;

    // preenche o vetor de inteiros com aleatórios
    for (i = 0; i < VETSIZE; i++)
        vetor[i] = random() % 1000 ;

    // escreve o vetor
    for (i = 0; i < VETSIZE; i++)
        printf ("%d ", vetor[i]) ;
    printf ("\n") ;

    // ordena o vetor (man qsort)
    // Protótipo: int (*compara_int) (const void *, const void *)
    qsort (vetor, VETSIZE, sizeof (int), compara_int) ;

    // escreve o vetor
    for (i = 0; i < VETSIZE; i++)
        printf ("%d ", vetor[i]) ;
    printf ("\n") ;
}
```

Conversão de/para strings

No caso específico de strings, a conversão destas para outros tipos é efetuada através de funções específicas:

```
#include <stdlib.h>

// string to float, int, long, long long
double  atof  (const char *str) ;
int     atoi  (const char *str) ;
long    atol  (const char *str) ;
long long atoll (const char *str) ;

// string to double, float, long double, com teste de erro
double strtod (const char *nptr, char **endptr) ;
float  strttof (const char *nptr, char **endptr) ;
long   double strtold (const char *nptr, char **endptr) ;
```

Conversão de/para strings

No sentido **número** -> **string**, a forma mais simples de converter um dado de qualquer tipo para string é usando a função **sprintf**, que formata e “imprime” o dado em uma string, de forma similar ao que a função printf realiza na saída padrão:

```
#include <stdio.h>

int main ()
{
    char buffer[256] ;
    float x ;

    x = 32.4 / 7 ;

    sprintf (buffer, "%5.4f", x) ; // "imprime" x na string buffer

    printf ("%s\n", buffer) ;

    return 0 ;
}
```

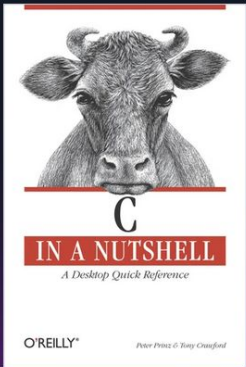
Conversão em C

Referência completa [AQUI](#)

O'REILLY [Explore Skills](#) [Features](#) [Radar Blog](#) [Buy Courses](#)

Q Search for books, courses, events, and more

[Plans](#) [Sign In](#)



BOOK

C in a Nutshell

by [Peter Prinz](#), [Tony Crawford](#)

 December 2005

 Beginner To Intermediate

 618 Pages

 20h 19m

 English

 O'Reilly Media, Inc.

[Read now](#)
Unlock full access

Content preview from [C in a Nutshell](#)

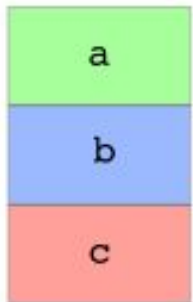
Chapter 4. Type Conversions

Uniãoes

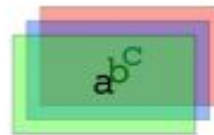
Uma **união (union)** é um tipo especial de estrutura (**struct**) no qual todos os campos internos são sobrepostos e ocupam a mesma posição na memória. Com isso, uma mesma posição na memória pode ser “vista” de diversas formas, conforme o campo da união que for acessado.

Struct ≠ Union

```
struct s1  
{  
    int    a ;  
    float  b ;  
    char   c ;  
} ;
```



```
union u1  
{  
    int    a ;  
    float  b ;  
    char   c ;  
} ;
```



Uniãos

Uniãos são uma forma eficiente de **armazenar valores de diferentes tipos em uma mesma posição de memória**. Obviamente, somente um valor pode ser armazenado a cada instante. A quantidade de memória ocupada por uma união corresponde ao **tamanho de seu maior campo**.

No exemplo abaixo, uma união permite “enxergar” os bytes individuais de um inteiro sem necessidade de operações de bits ou aritmética de ponteiros:

```
// guarda um inteiro OU um vetor de 4 bytes
typedef union
{
    int value ;
    unsigned char byte[sizeof(int)] ;
} intParts ;
```

```
int main ()
{
    intParts a ;
    int i ;

    a.value = 653459 ;

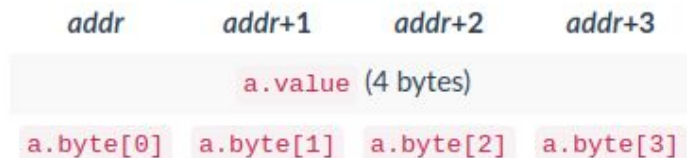
    printf ("%d: ", a.value) ;

    for (i = 0; i < sizeof (int); i++)
        printf ("%02x ", a.byte[i]) ;
    printf ("\n") ;
}
```


Unões

Unões são uma forma eficiente de **armazenar valores de diferentes tipos em uma mesma posição de memória** a cada instante. A qual **depende do tamanho de seu maior**

Os campos `a.value` e `a.byte[]` estão alocados da seguinte forma na memória:



No exemplo abaixo, um `int` é armazenado em um `union` sem necessidade de operações de bits ou aritmética de ponteiros:

```
// guarda um inteiro OU um vetor de 4 bytes
typedef union
{
    int value ;
    unsigned char byte[sizeof(int)] ;
} intParts ;
```

```
int main ()
{
    intParts a ;
    int i ;

    a.value = 653459 ;

    printf ("%d: ", a.value) ;

    for (i = 0; i < sizeof (int); i++)
        printf ("%02x ", a.byte[i]) ;
    printf ("\n") ;
}
```

Uniãos

Uniãos podem ser usadas para armazenar valores de diversos tipos em uma única localização de memória. Por exemplo, a união a seguir pode ser usada para armazenar números de diversos tipos:

```
typedef union
{
    short  shortVal ;
    int    intVal   ;
    long   longVal  ;
    float  floatVal ;
    double doubleVal ;
} numeric_t ;
```

Entretanto, como saber qual o tipo do último valor armazenado, ou seja, qual o valor corrente?

Se armazenarmos um `int` e tentarmos ler um `float` teremos um valor errado, pois os valores são lidos byte a byte diretamente da área de memória da união, sem conversões.

Unões

```
typedef union
{
    short  shortVal ;
    int    intVal ;
    long   longVal ;
    float  floatVal ;
    double doubleVal ;
} numeric_t ;
```

```
int main ()
{
    numeric_t a ;

    a.shortVal = 741 ;
    printf ("short : %d\n", a.shortVal) ;
    printf ("float : %f\n\n", a.floatVal) ;

    a.floatVal = 327.5432 ;
    printf ("float : %f\n", a.floatVal) ;
    printf ("short : %d\n\n", a.shortVal) ;

    a.doubleVal = 327.5432 ;
    printf ("double: %lf\n", a.doubleVal) ;
    printf ("float : %f\n", a.floatVal) ;
    printf ("short : %d\n", a.shortVal) ;
    return 0 ;
}
```

diego@diego:~/Downloads\$./0

short : 741

float : 0.000000

float : 327.543213

short : -14968

double: 327.543200

float : -4985416349728137472163023159296.000000

short : -19713

Para resolver esse problema pode ser usada uma estrutura contendo a união e uma variável que indique o tipo do último valor armazenado:

```
typedef struct
{
    union          // ATTENTION: "anonymous" union
    {
        short  shortVal ;
        int    intVal ;
        long   longVal ;
        float  floatVal ;
        double doubleVal ;
    } ;
    enum { SHORT, INT, LONG, FLOAT, DOUBLE } type ;
} numeric_t ;

// imprime tipo numérico
void print_num (numeric_t n)
{
    switch (n.type)
    {
        case SHORT : printf ("%d", n.shortVal) ; break ;
        case INT   : printf ("%d", n.intVal)   ; break ;
        case LONG  : printf ("%ld", n.longVal)  ; break ;
        case FLOAT : printf ("%f", n.floatVal)  ; break ;
        case DOUBLE: printf ("%lf", n.doubleVal); break ;
        default    : printf ("NaN") ;
    }
}
```

```
int main ()
{
    numeric_t a ;

    a.shortVal = 119 ;
    a.type = SHORT ;
    print_num (a) ;
    printf ("\n") ;

    a.longVal = 3451212796756 ;
    a.type = LONG ;
    print_num (a) ;
    printf ("\n") ;

    a.doubleVal = 3.141592653589793 ;
    a.type = DOUBLE ;
    print_num (a) ;
    printf ("\n") ;
}
```

Uniãoes

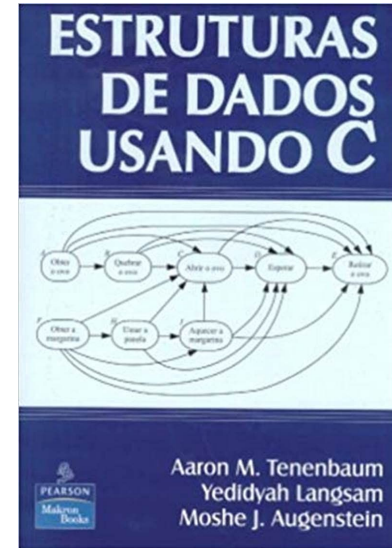
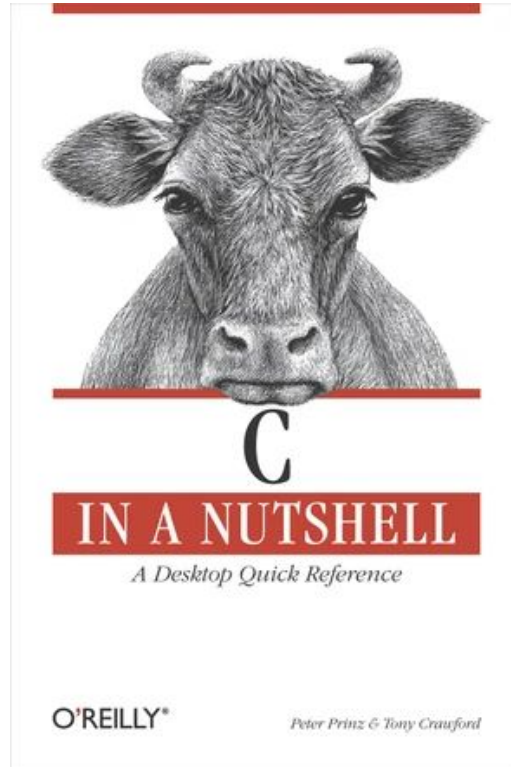
A linguagem C respeita a ordem de declaração dos campos de uma estrutura, ou seja, eles são colocados na memória na mesma ordem em que são declarados.

Isso permite outro exemplo interessante do uso de uma união, no qual as moedas podem ser acessadas com nomes individuais ou como elementos de um vetor:

```
typedef union {  
    struct  
    {  
        int quarter;  
        int dime;  
        int nickel;  
        int penny;  
    };  
    int coins[4];  
} Coins_t ;
```

```
Coins_t a ;  
  
a.dime = 34 ;           // são operações equivalentes  
a.coins[1] = 34 ;
```

Referências



Unões

Exercício

Utilizando uma única variável union, crie uma função que receba um inteiro e calcule seu quadrado, em seguida, receba um caractere e, caso maiúsculo, imprima minúsculo, caso minúsculo, imprima maiúsculo, e por último, receba uma string de no máximo 8 caracteres e imprima seu inverso.